

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 3 LECTURE1

TOUR OF COMPUTER SYSTEMS

Tour of Computer Systems

- * A computer system is a collection of hardware and software components that work together to run computer programs.
- * Let us begin our study of systems by tracing the lifetime of the hello program, from the time it is created by a programmer, until it runs on a system, prints its simple message, and terminates.
- * As we follow the lifetime of the program, we will briefly introduce the key concepts, terminology, and components that come into play.

```
#include <stdio.h>

int main()
{
    printf("hello, world\n");
    return 0;
}
```

hello.c Program

Information is Bits + Context

- * Our hello program begins life as a *source program (or source file)* that the programmer creates with an editor and saves in a text file called `hello.c`.
- * The source program is a sequence of bits, each with a value of 0 or 1, organized in 8-bit chunks called *byte* where each byte represents some text character in the program.
- * Most modern systems represent text characters using the **ASCII standard** that represents each character with a unique byte-sized integer value.

#	i	n	c	l	u	d	e	<sp>	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	<sp>	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	<sp>	<sp>	<sp>	<sp>	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	<sp>	w	o	r	l	d	\	n	"	)	;	\n	}
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	125

ASCII text representation of `hello.c`

Information is Bits + Context

- * The `hello.c` program is stored in a file as a sequence of bytes where each byte has an integer value that corresponds to some character.
- * Notice that each text line is terminated by the invisible *newline* character `'\n'`, which is represented by the integer value 10.
- * Files such as `hello.c` that consist exclusively of ASCII characters are known as *text files*. All other files are known as *binary files*.

The representation of `hello.c` illustrates a fundamental idea:

- * All information in a system—including disk files, programs stored in memory, user data stored in memory, and data transferred across a network—is represented as a bunch of bits.
- * The only thing that distinguishes different data objects is the context in which we view them.
- * For example, in different contexts, the same sequence of bytes might represent an integer, floating-point number, character string, or machine instruction.

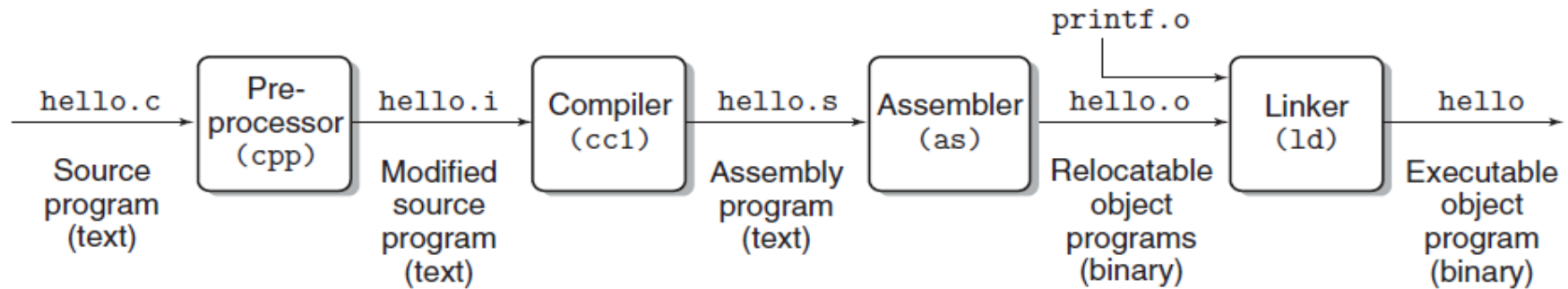
Compilation System

- * The individual C statements in `hello.c` high level C program must be translated into a sequence of low-level *machine-language* instructions.
- * These instructions are then packaged in a form called an *executable object program* and stored as a binary disk file.
- * Object programs are also referred to as *executable object files*.
- * On a Unix system, the translation from source file to object file is performed by a *compiler driver*:

```
unix> gcc -o hello hello.c
```

- * The gcc compiler driver reads the source file `hello.c` and translates it into an executable object file `hello`.
- * The translation is performed in the sequence of four phases - *preprocessor, compiler, assembler, and linker*
- * The programs that perform the four phases (*preprocessor, compiler, assembler, and linker*) are known collectively as the *compilation system*.

Compilation System



Compilation System

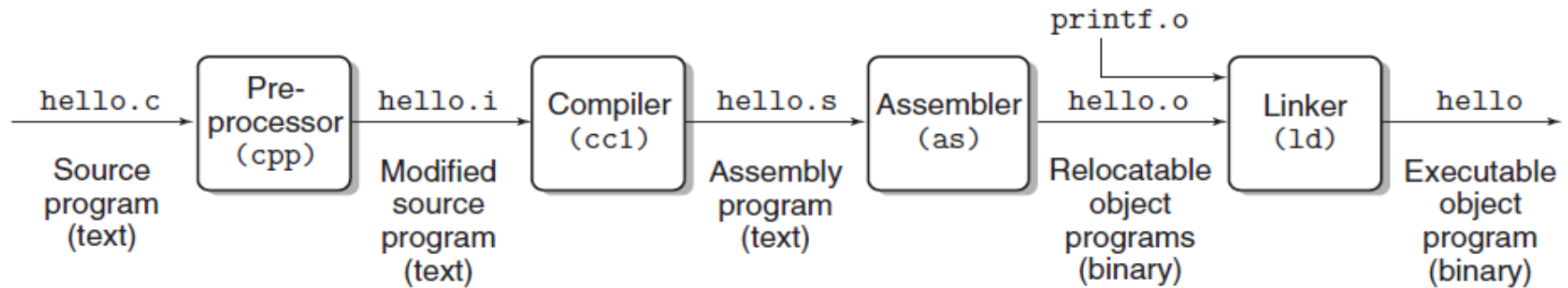
□ **Preprocessing phase**

- * The preprocessor (`cpp`) modifies the original C program according to directives that begin with the `#` character.
- * For example, the `#include <stdio.h>` command in line 1 of `hello.c` tells the pre-processor to read the contents of the system header file `stdio.h` and insert it directly into the program text. The result is another C program, typically with the `.i` suffix.

□ **Compilation phase**

- * The compiler (`cc1`) translates the text file `hello.i` into the text file `hello.s`, which contains an *assembly-language program*.
- * Each statement in an assembly-language program exactly describes one low-level machine-language instruction in a standard text form. Assembly language is useful because it provides a common output language for different compilers for different high-level languages..

Compilation System



Compilation System

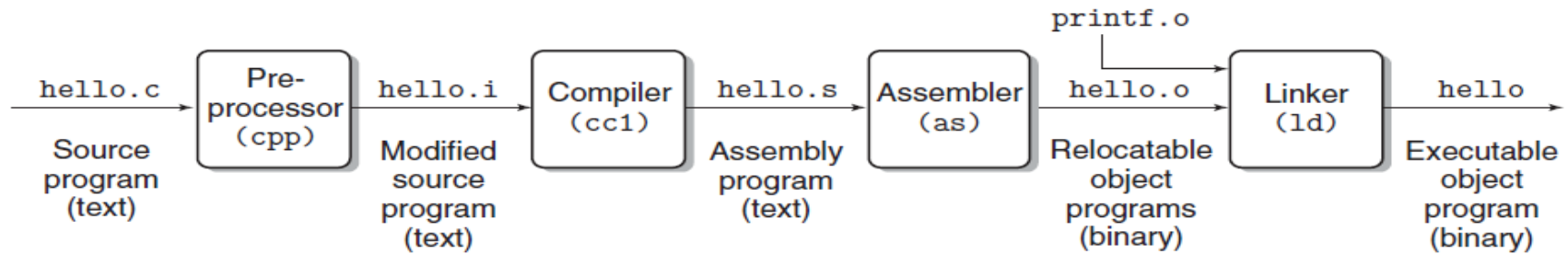
□ **Assembly phase**

- * The assembler (as) translates hello.s into machine language instructions, packages them in a form known as a relocatable object program, and stores the result in the object file hello.o.
- * The hello.o file is a binary file whose bytes encode machine language instructions rather than characters.

□ **Linking phase**

- * hello program calls the printf function, which is part of the *standard C library* provided by every C compiler. The printf function resides in a separate precompiled object file called printf.o, which must somehow be merged with our hello.o program. The linker (ld) handles this merging.
- * The result is the hello file, which is an *executable object file* (or simply *executable*) that is ready to be loaded into memory and executed by the system.

Compilation System

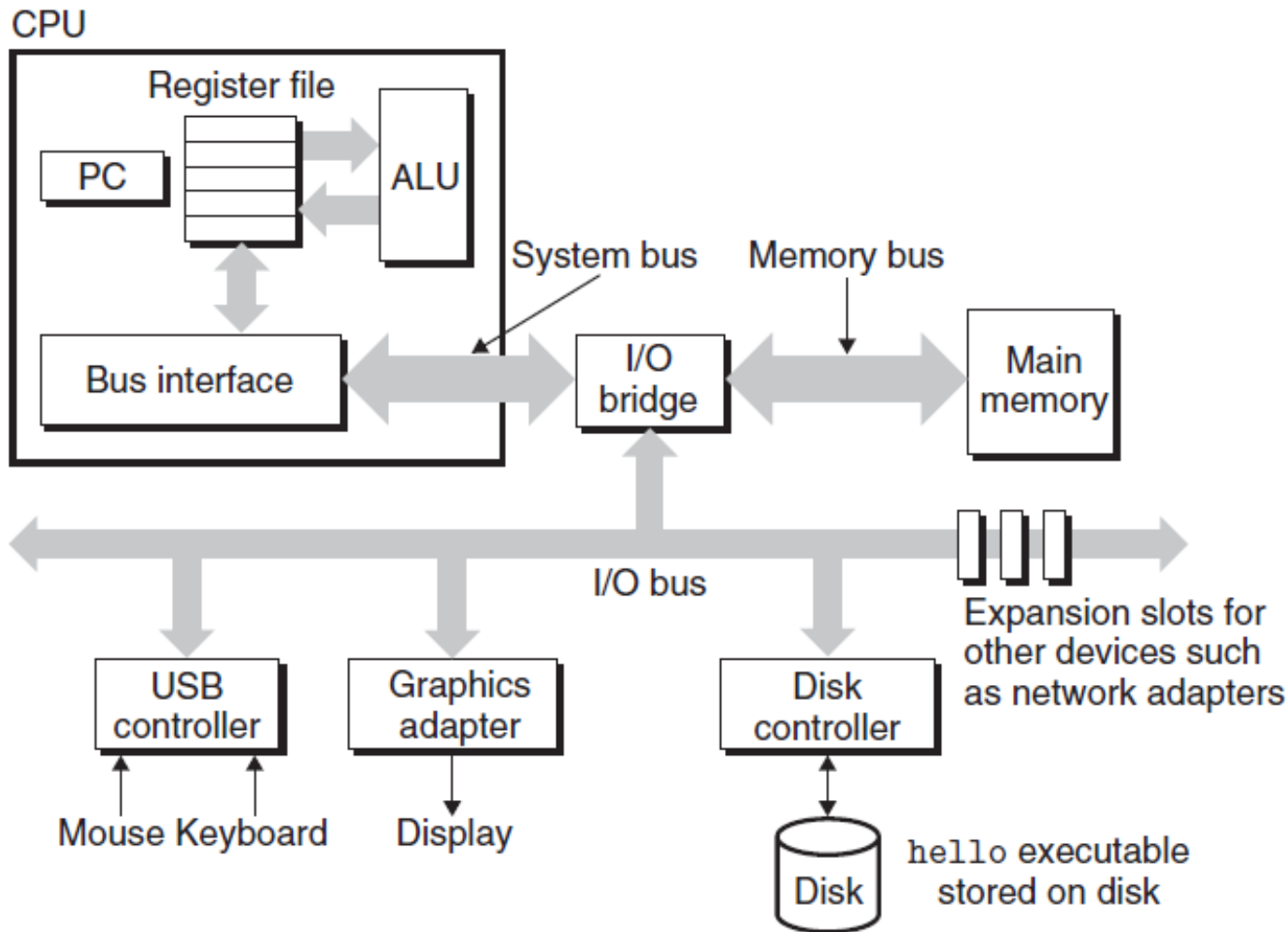


Compilation System

- * hello.c source program has been translated by the compilation system into an executable object file called hello that is stored on disk.
- * To run the executable file on a Unix system, we type its name to an application program known as a *shell*:

```
unix> ./hello  
hello, world  
unix>
```
- * The shell is a command-line interpreter that prints a prompt, waits for you to type a command line, and then performs the command.
- * If the first word of the command line does not correspond to a built-in shell command, then the shell assumes that it is the name of an executable file that it should load and run.
- * So in this case, the shell loads and runs the hello program and then waits for it to terminate.
- * The hello program prints its message to the screen and then terminates. The shell then prints a prompt and waits for the next input command line.

Hardware Organization



Hardware Organization of a typical Computer System

Hardware Organization

★ Buses

- Buses is a collection of electrical conduits running throughout the system that carry bytes of information back and forth between the components.
- Buses are typically designed to transfer fixed-sized chunks of bytes known as words.
- The number of bytes in a word (the word size) is a fundamental system parameter that varies across systems.
- Most machines today have word sizes of either 4 bytes (32 bits) or 8 bytes (64 bits).
- Assume that a word is of size 4 bytes, and that the buses transfer only one word at a time

Hardware Organization

★ I/O Devices

- Input/output (I/O) devices are the system's connection to the external world.
- Our system has four I/O devices: a keyboard and mouse for user input, a display for user output, and a disk drive (or simply disk) for long-term storage of data and programs.
- Initially, the executable hello program resides on the disk.
- Each I/O device is connected to the I/O bus by either a controller or an adapter.
- The distinction between the two is mainly one of packaging.
- Controllers are chip sets in the device itself or on the system's main printed circuit board (often called the motherboard).
- An adapter is a card that plugs into a slot on the motherboard.
- Regardless, the purpose of each is to transfer information back and forth between the I/O bus and an I/O device.

Hardware Organization

★ Main Memory

- The main memory is a temporary storage device that holds both a program and the data it manipulates while the processor is executing the program.
- Physically, main memory consists of a collection of dynamic random access memory(DRAM) chips.
- Logically, memory is organized as a linear array of bytes, each with its own unique address (array index) starting at zero.
- In general, each of the machine instructions that constitute a program can consist of a variable number of bytes.
- The sizes of data items that correspond to C program variables vary according to type.
- For example, on an IA32 machine running Linux, data of type short requires two bytes, types int, float, and long four bytes, and type double eight bytes.

Hardware Organization

★ Processor

- The central processing unit (CPU), or simply processor, is the engine that interprets (or executes) instructions stored in main memory.
- At its core is a word-sized storage device (or register) called the program counter (PC).
- At any point in time, the PC points at (contains the address of) some machine-language instruction in main memory.
- From the time that power is applied to the system, until the time that the power is shut off, a processor repeatedly executes the instruction pointed at by the program counter and updates the program counter to point to the next instruction.
- A processor appears to operate according to a very simple instruction execution model, defined by its instruction set architecture.
- In this model, instructions execute in strict sequence, and executing a single instruction involves performing a series of steps.
- The processor reads the instruction from memory pointed at by the program counter (PC) (Fetch operation), interprets the bits in the instruction (Decode operation), performs some simple operation dictated by the instruction (Execute Operation), and then updates the PC to point to the next instruction, which may or may not be contiguous in memory to the instruction that was just executed.

Hardware Organization

★ Processor

- There are only a few of these simple operations, and they revolve around main memory, the register file, and the arithmetic/logic unit (ALU).
- The register file is a small storage device that consists of a collection of word-sized registers, each with its own unique name.
- The ALU computes new data and address values.
- Here are some examples of the simple operations that the CPU might carry out at the request of an instruction:
- Load: Copy a byte or a word from main memory into a register, overwriting the previous contents of the register.
- Store: Copy a byte or a word from a register to a location in main memory, overwriting the previous contents of that location.
- Operate: Copy the contents of two registers to the ALU, perform an arithmetic operation on the two words, and store the result in a register, overwriting the previous contents of that register.
- Jump: Extract a word from the instruction itself and copy that word into the program counter (PC), overwriting the previous value of the PC.

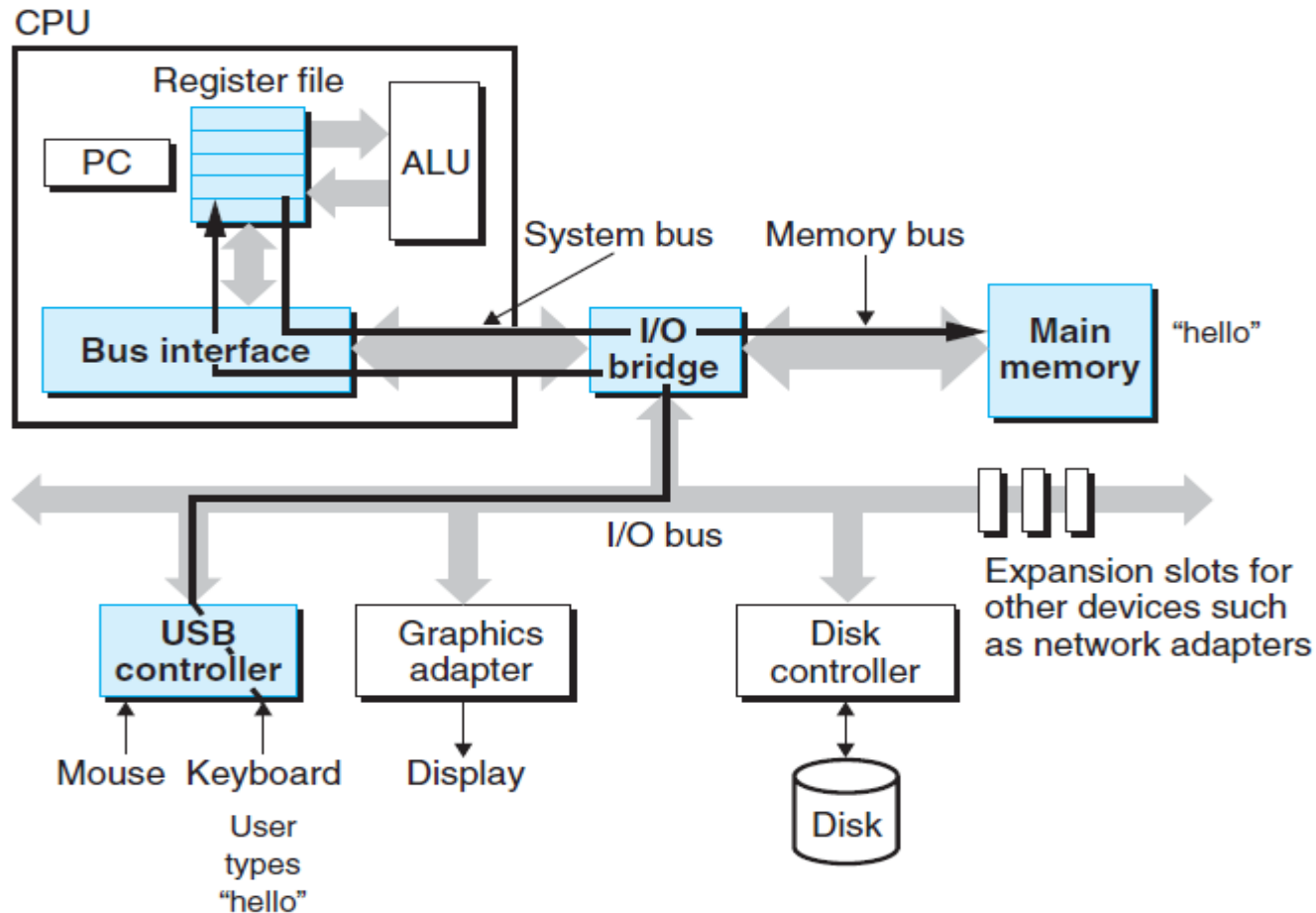
Hardware Organization

★ Processor

- There are only a few of these simple operations, and they revolve around main memory, the register file, and the arithmetic/logic unit (ALU).
- **The register file is a small storage device that consists of a collection of word-sized registers, each with its own unique name.**
- **The ALU computes new data and address values.**
- Here are some examples of the simple operations that the CPU might carry out at the request of an instruction:
- **Load:** Copy a byte or a word from main memory into a register, overwriting the previous contents of the register.
- **Store:** Copy a byte or a word from a register to a location in main memory, overwriting the previous contents of that location.
- **Operate:** Copy the contents of two registers to the ALU, perform an arithmetic operation on the two words, and store the result in a register, overwriting the previous contents of that register.
- **Jump:** Extract a word from the instruction itself and copy that word into the program counter (PC), overwriting the previous value of the PC.

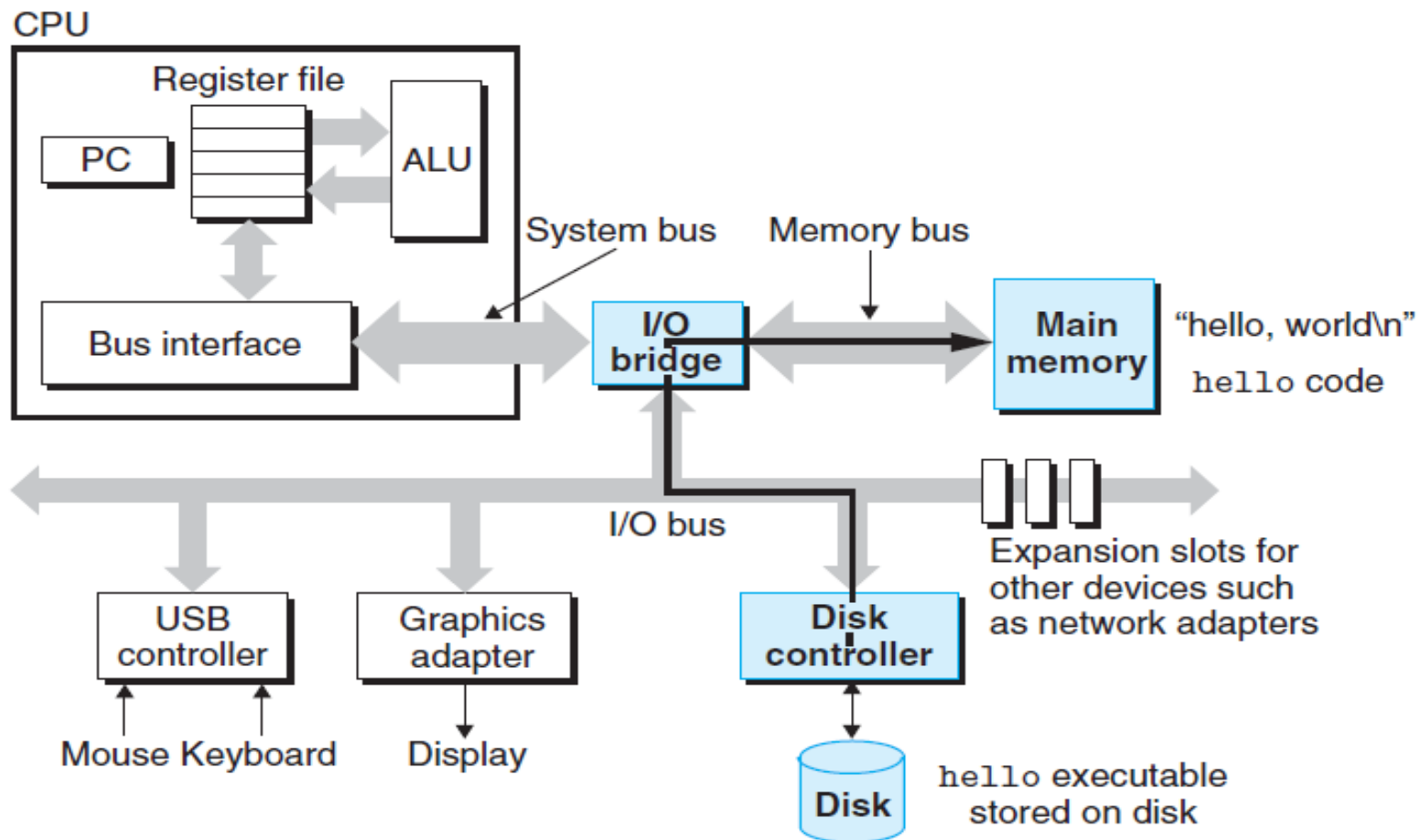
Running the hello Program

Initially, the shell program is executing its instructions, waiting for us to type a command. As we type the characters “./hello” at the keyboard, the shell program reads each one into a register, and then stores it in memory



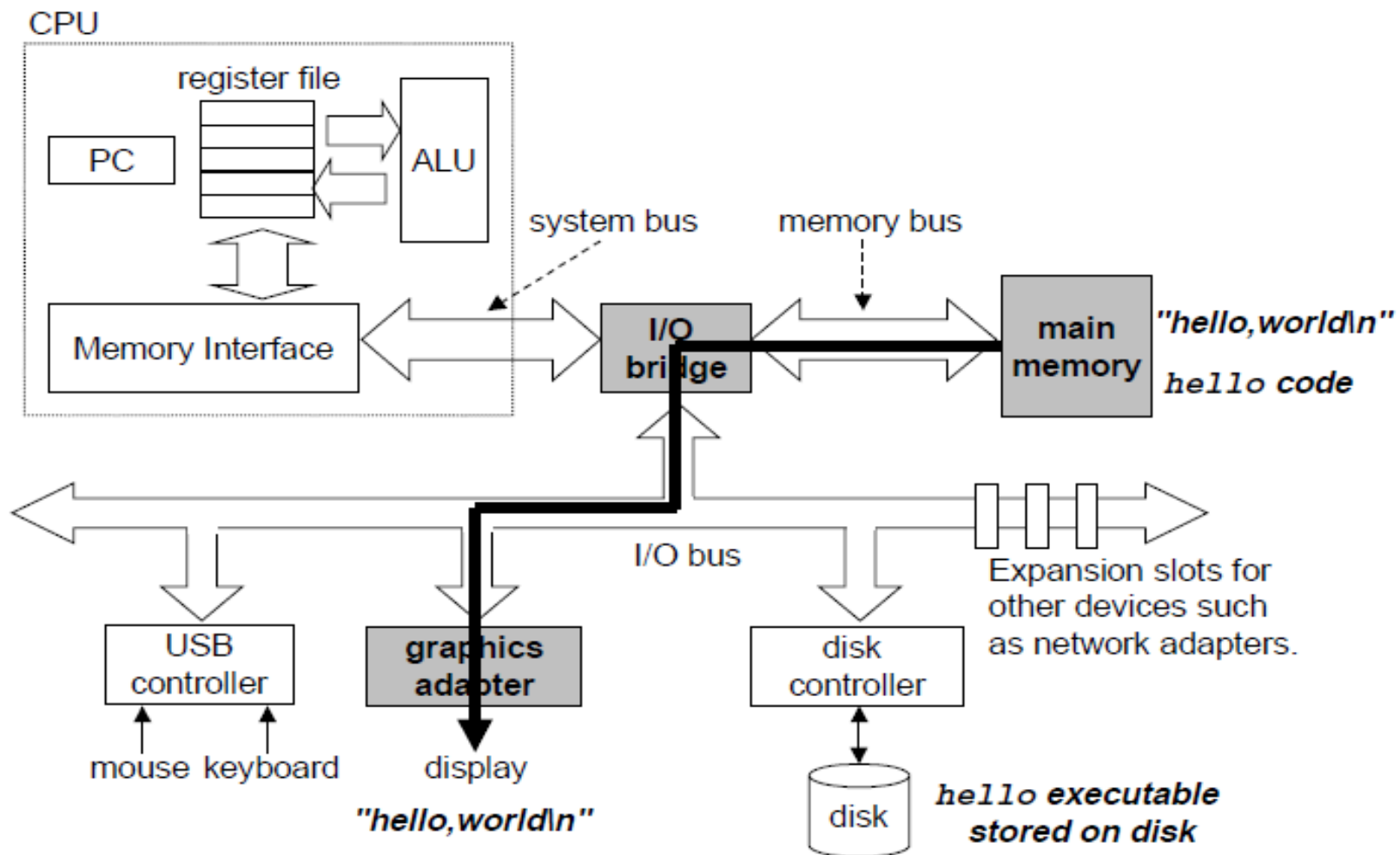
Running the hello Program

When we hit the enter key on the keyboard, the shell knows that we have finished typing the command. The shell then loads the executable hello file by executing a sequence of instructions that copies the code and data in the hello object file from disk to main memory. The data include the string of characters “hello, world\n” that will eventually be printed out. Using a technique known as direct memory access (DMA), the data travels directly from disk to main memory, without passing through the processor.



Running the hello Program

Once the code and data in the hello object file are loaded into memory, the processor begins executing the machine-language instructions in the hello program's main routine. These instructions copy the bytes in the "hello, world\n" string from memory to the register file, and from there to the display device, where they are displayed on the screen.



Writing the output string from memory to the display

Need for Cache Memories

- As it is evident that system spends a lot of time moving information from one place to another.
- The machine instructions in the hello program are originally stored on disk.
- When the program is loaded, they are copied to main memory.
- As the processor runs the program, instructions are copied from main memory into the processor.
- Similarly, the data string “hello,world\n”, originally on disk, is copied to main memory, and then copied from main memory to the display device.
- **From a programmer’s perspective, much of this copying is overhead that slows down the “real work” of the program.**
- **Thus, a major goal for system designers is to make these copy operations run as fast as possible.**

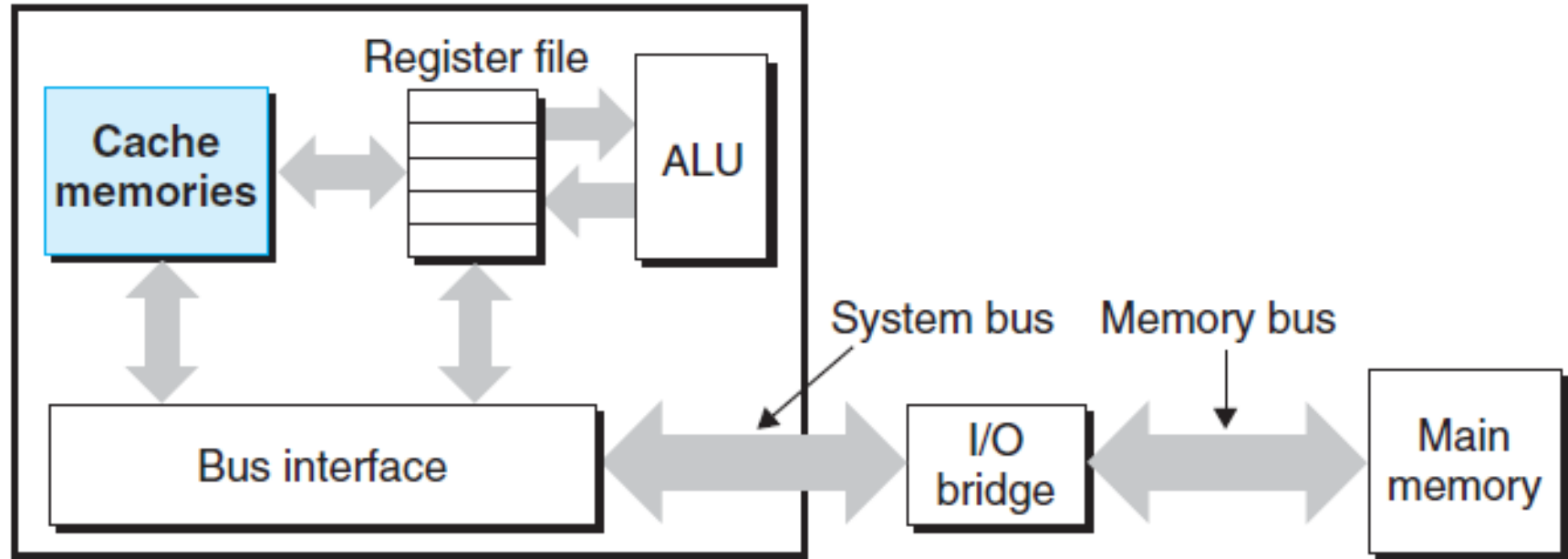
Need for Cache Memories

- Because of physical laws, larger storage devices are slower than smaller storage devices. And faster devices are more expensive to build than their slower counterparts.
- For example, the disk drive on a typical system might be 1000 times larger than the main memory, but it might take the processor 10,000,000 times longer to read a word from disk than from memory.
- Similarly, a typical register file stores only a few hundred bytes of information, as opposed to billions of bytes in the main memory.
- However, the processor can read data from the register file almost 100 times faster than from memory.
- Even more troublesome, as semiconductor technology progresses over the years, this processor-memory gap continues to increase.
- It is easier and cheaper to make processors run faster than it is to make main memory run faster.

Cache Memories

To deal with the processor-memory gap, system designers include smaller faster storage devices called **cache memories** (or simply **caches**) that serve as temporary staging areas for information that the processor is likely to need in the near future.

CPU chip



Cache Memories

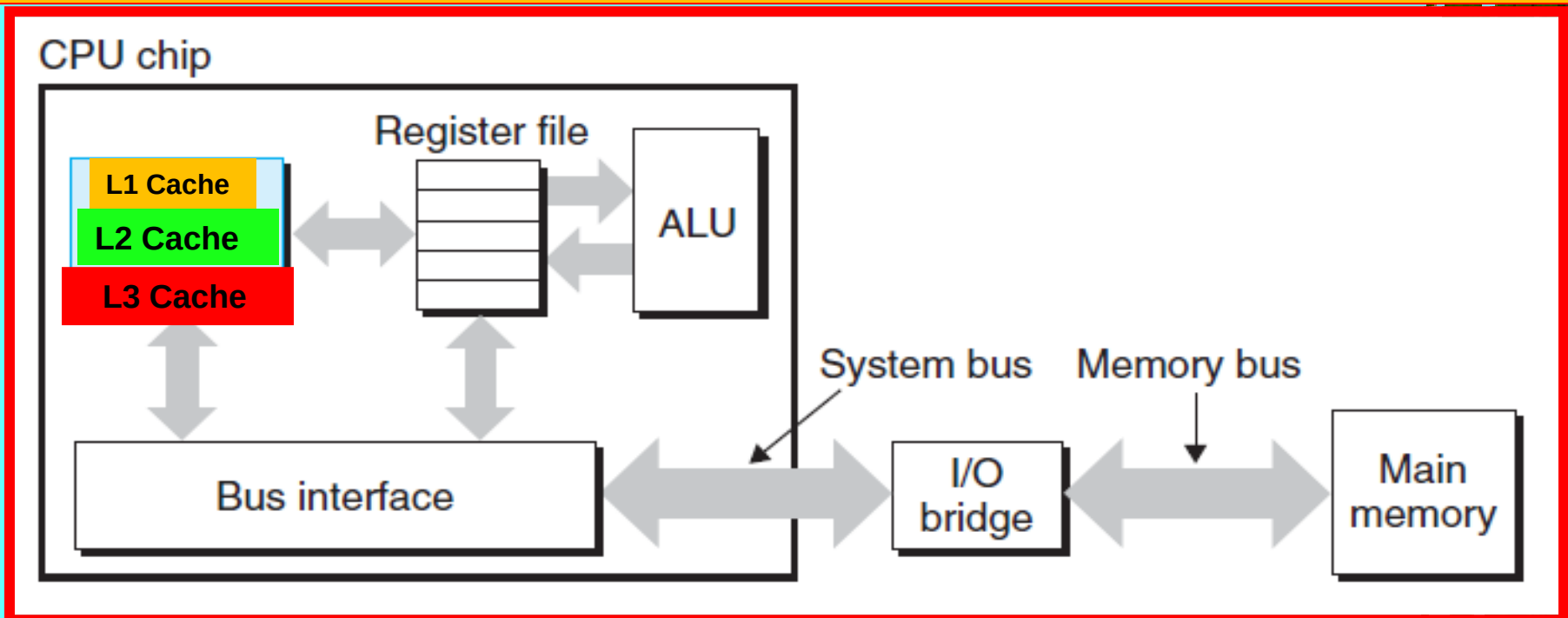
L1 cache on the processor chip holds tens of thousands of bytes and can be accessed nearly as fast as the register file. A larger L2 cache with hundreds of thousands to millions of bytes is connected to the processor by a special bus. It might take 5 times longer for the process to access the L2 cache than the L1 cache, but this is still 5 to 10 times faster than accessing the main memory.

The L1 and L2 caches are implemented with a hardware technology known as static random access memory (SRAM).

Newer and more powerful systems even have three levels of cache: L1, L2, and L3.

The idea behind caching is that a system can get the effect of both a very large memory and a very fast one by exploiting locality, the tendency for programs to access data and code in localized regions.

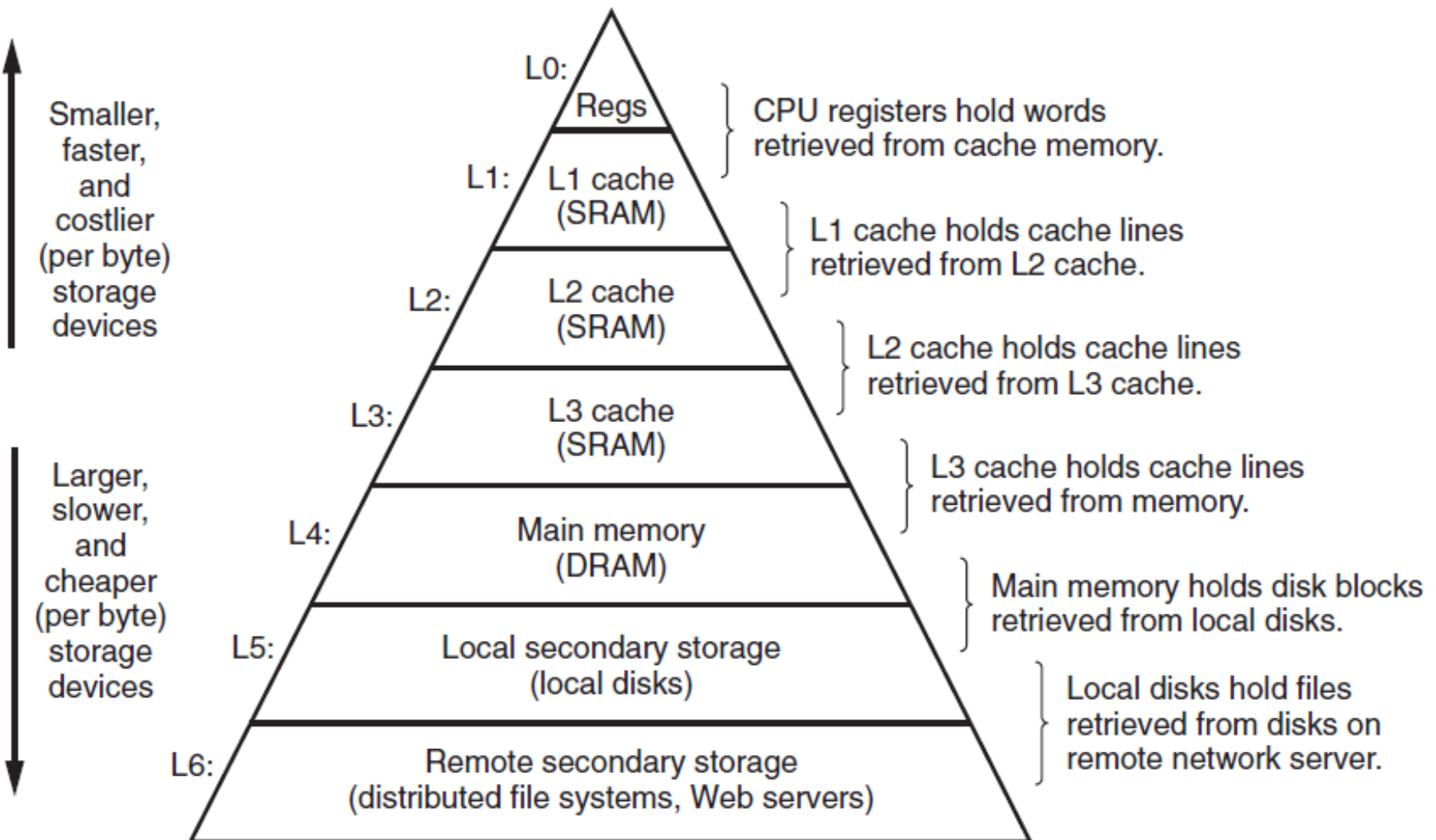
By setting up caches to hold data that is likely to be accessed often, we can perform most memory operations using the fast caches.



OPERATING SYSTEM

- When the shell loaded and ran the hello program, and when the hello program printed its message, these programs relied on the services provided by the operating system to access the keyboard, display, disk, or main memory directly.
- The operating system can be thought as a layer of software interposed between the application program and the hardware
- All attempts by an application program to manipulate the hardware must go through the operating system.
- The operating system has two primary purposes:
 - To protect the hardware from misuse by runaway applications, and
 - To provide applications with simple and uniform mechanisms for manipulating complicated and often wildly different low-level hardware devices.
- ❖ The operating system achieves both goals via the fundamental abstractions : processes, virtual memory, and files.
- ❖ Files are abstractions for I/O devices, Virtual memory is an abstraction for both the main memory and disk I/O devices, and processes are abstractions for the processor, main memory, and I/O devices.

Storage Devices form a Hierarchy

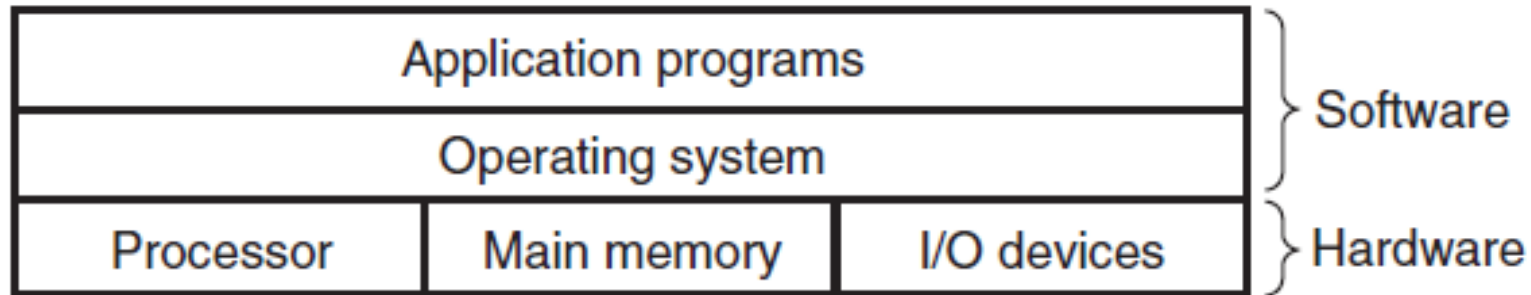


An Example of a Memory Hierarchy

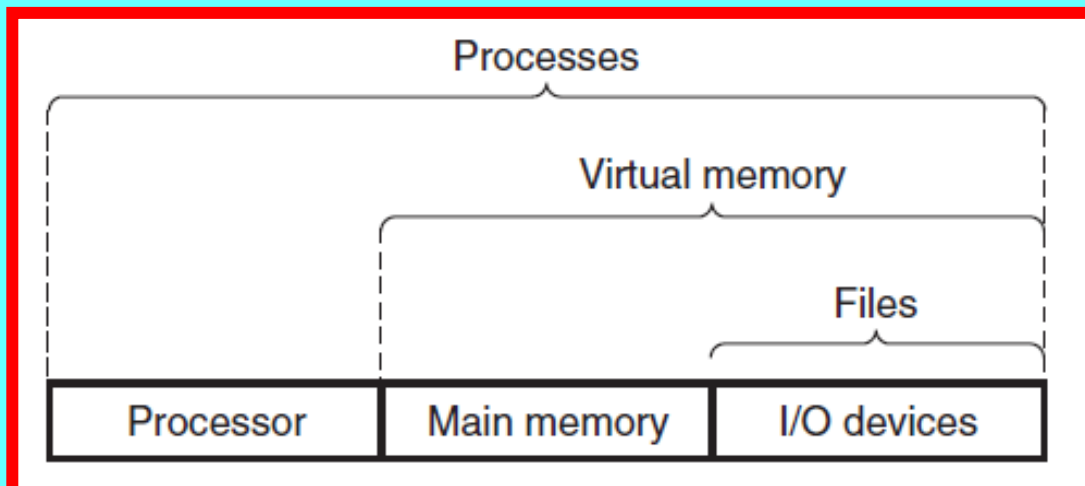
Storage Devices form a Hierarchy

- The notion of inserting a smaller, faster storage device (e.g., cache memory) between the processor and a larger slower device (e.g., main memory) turns out to be a general idea.
- In fact, the storage devices in every computer system are organized as a memory hierarchy.
- As we move from the top of the hierarchy to the bottom, the devices become slower, larger, and less costly per byte.
- The register file occupies the top level in the hierarchy, which is known as level 0, or L0.
- We show three levels of caching L1 to L3, occupying memory hierarchy levels 1 to 3, Main memory occupies level 4, and so on.
- The main idea of a memory hierarchy is that storage at one level serves as a cache for storage at the next lower level.
- Thus, the register file is a cache for the L1 cache. Caches L1 and L2 are caches for L2 and L3, respectively.
- The L3 cache is a cache for the main memory, which is a cache for the disk.
- On some networked systems with distributed file systems, the local disk serves as a cache for data stored on the disks of other systems.

Operating System



Layered View of a Computer System



Abstraction provided by an Operating System

Process

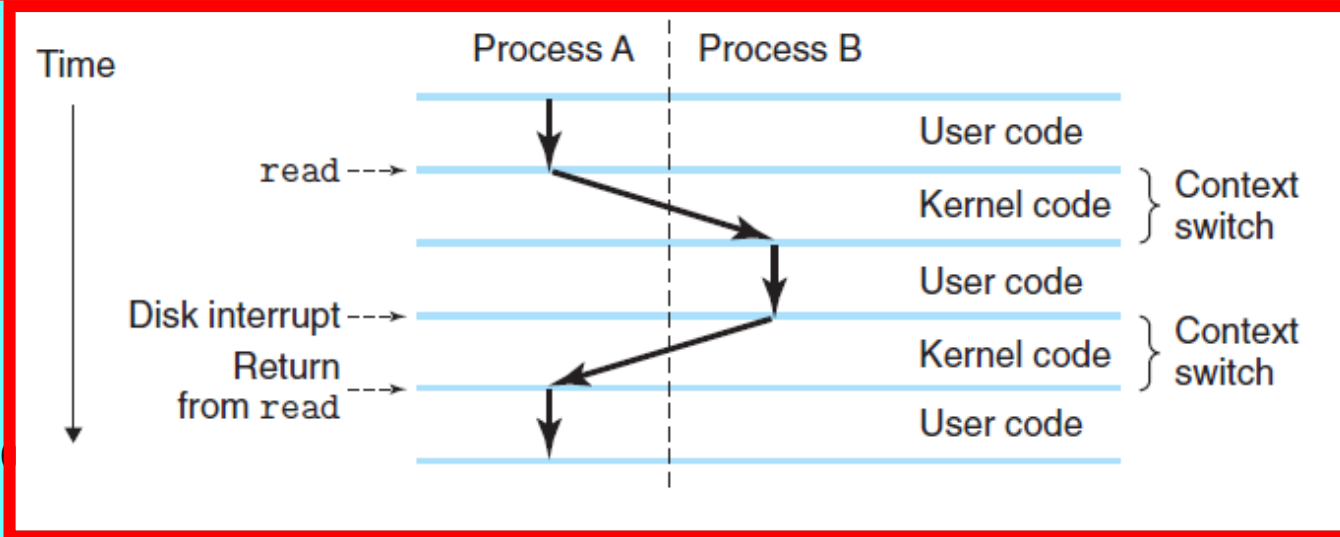
- When a program such as hello runs on a modern system, the operating system provides the illusion that the program is the only one running on the system.
- The program appears to have exclusive use of both the processor, main memory, and I/O devices.
- The processor appears to execute the instructions in the program, one after the other, without interruption and the code and data of the program appear to be the only objects in the system's memory.
- These illusions are provided by the notion of a process, one of the most important and successful ideas in computer science.
- A process is the operating system's abstraction for a running program.
- Multiple processes can run concurrently on the same system, and each process appears to have exclusive use of the hardware.
- By concurrently, we mean that the instructions of one process are interleaved with the instructions of another process.
- In most systems, there are more processes to run than there are CPUs to run them.
- Traditional systems could only execute one program at a time, while newer multicore processors can execute several programs simultaneously. In either case, a single CPU can appear to execute multiple processes concurrently by having the processor switch among them. The operating system performs this interleaving with a mechanism known as context switching.

Process Context Switching

- Consider only a uniprocessor system containing a single CPU.
- The operating system keeps track of all the state information that the process needs in order to run.
- This state, which is known as the context, includes information such as the current values of the PC, the register file, and the contents of main memory.
- At any point in time, a uniprocessor system can only execute the code for a single process.
- When the operating system decides to transfer control from the current process to some new process, it performs a context switch by saving the context of the current process, restoring the context of the new process, and then passing control to the new process.
- The new process picks up exactly where it left off.

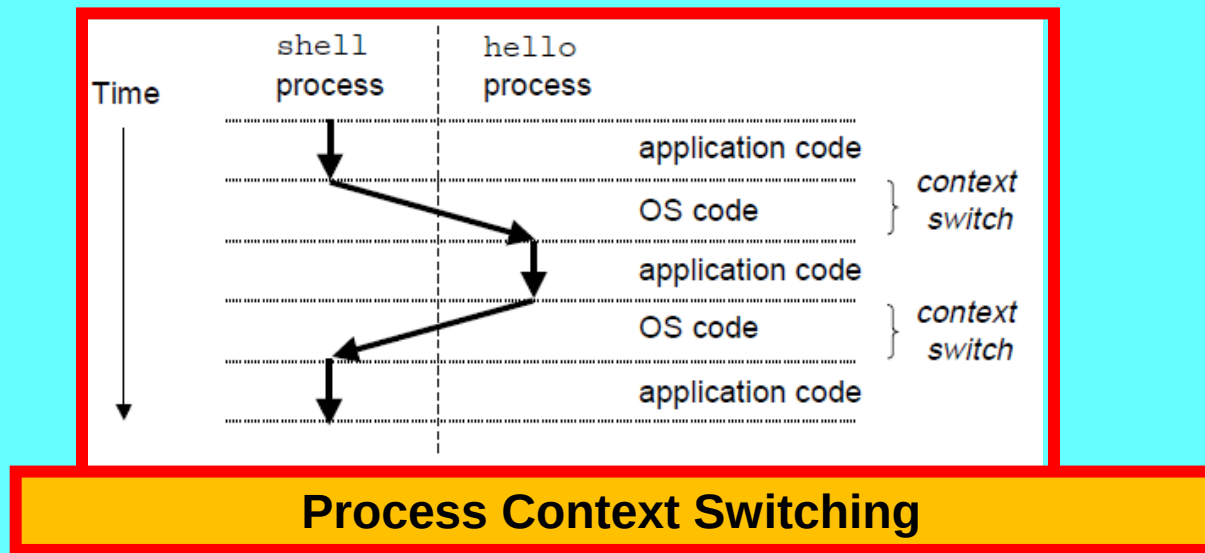
Process Context Switching

- There are two concurrent processes in our hello example scenario: the shell process and the hello process.
- Initially, the shell process is running alone, waiting for input on the command line.
- When we ask it to run the hello program, the shell carries out our request by invoking a special function known as a system call that passes control to the operating system.
- The operating system saves the shell's context, creates a new hello process and its context, and then passes control to the new hello process.
- After hello terminates, the operating system restores the context of the shell process and passes control back to it, where it waits for the next command line input.
- Implementing the process abstraction requires close cooperation between both the low-level hardware and the operating system software.



Process Context Switching

hello program



THREADS

- Although we normally think of a process as having a single control flow, in modern systems a process can actually consist of multiple execution units, called threads, each running in the context of the process and sharing the same code and global data.
- Threads are an increasingly important programming model because of the requirement for concurrency in network servers, because it is easier to share data between multiple threads than between multiple processes, and because threads are typically more efficient than processes.
- Multi-threading is also one way to make programs run faster when multiple processors are available

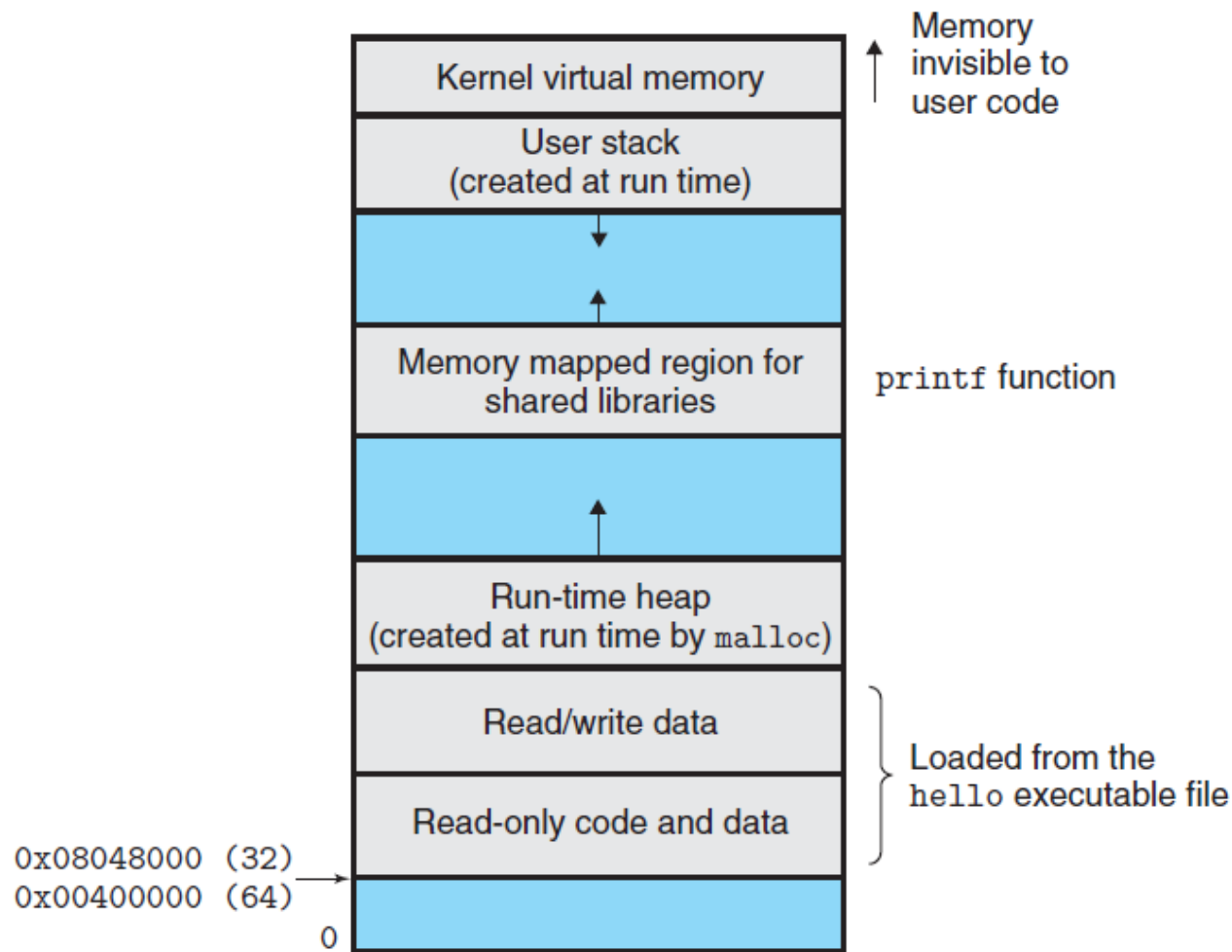
Virtual Memory

- Virtual memory is an abstraction that provides each process with the illusion that it has exclusive use of the main memory.
- Each process has the same uniform view of memory, which is known as its virtual address space.
- In Linux, the topmost region of the address space is reserved for code and data in the operating system that is common to all processes.
- The lower region of the address space holds the code and data defined by the user's process.
- Note that addresses increase from the bottom to the top.

Virtual Memory

- The **virtual address space** seen by each process in Linux consists of a number of well defined areas, each with a specific purpose.
- ❑ **Program code and data area**
- ❑ **Heap area**
- ❑ **Shared Libraries area**
- ❑ **Stack area**
- ❑ **Kernel Virtual Memory area**

Virtual Memory



Process Virtual Address Space

Virtual Memory

□ Program code and data area

- ★ Code begins at the same fixed address for all processes, followed by data locations that correspond to global C variables.
- ★ The code and data areas are initialized directly from the contents of an executable object file, in our case the hello executable.

□ Heap area

- ❖ The code and data areas are followed immediately by the run-time heap.
- ❖ Unlike the code and data areas, which are fixed in size once the process begins running, the heap expands and contracts dynamically at run time as a result of calls to C standard library routines such as malloc and free.

Virtual Memory

❑ Shared Libraries area

- ❖ Near the middle of the address space is an area that holds the code and data for shared libraries such as the C standard library and the math library.
- ❖ The notion of a shared library is a powerful but somewhat difficult concept.

❑ Stack area

- ❖ At the top of the user's virtual address space is the user stack that the compiler uses to implement function calls.
- ❖ Like the heap, the user stack expands and contracts dynamically during the execution of the program.
- ❖ In particular, each time we call a function, the stack grows. Each time we return from a function, it contracts.

Virtual Memory

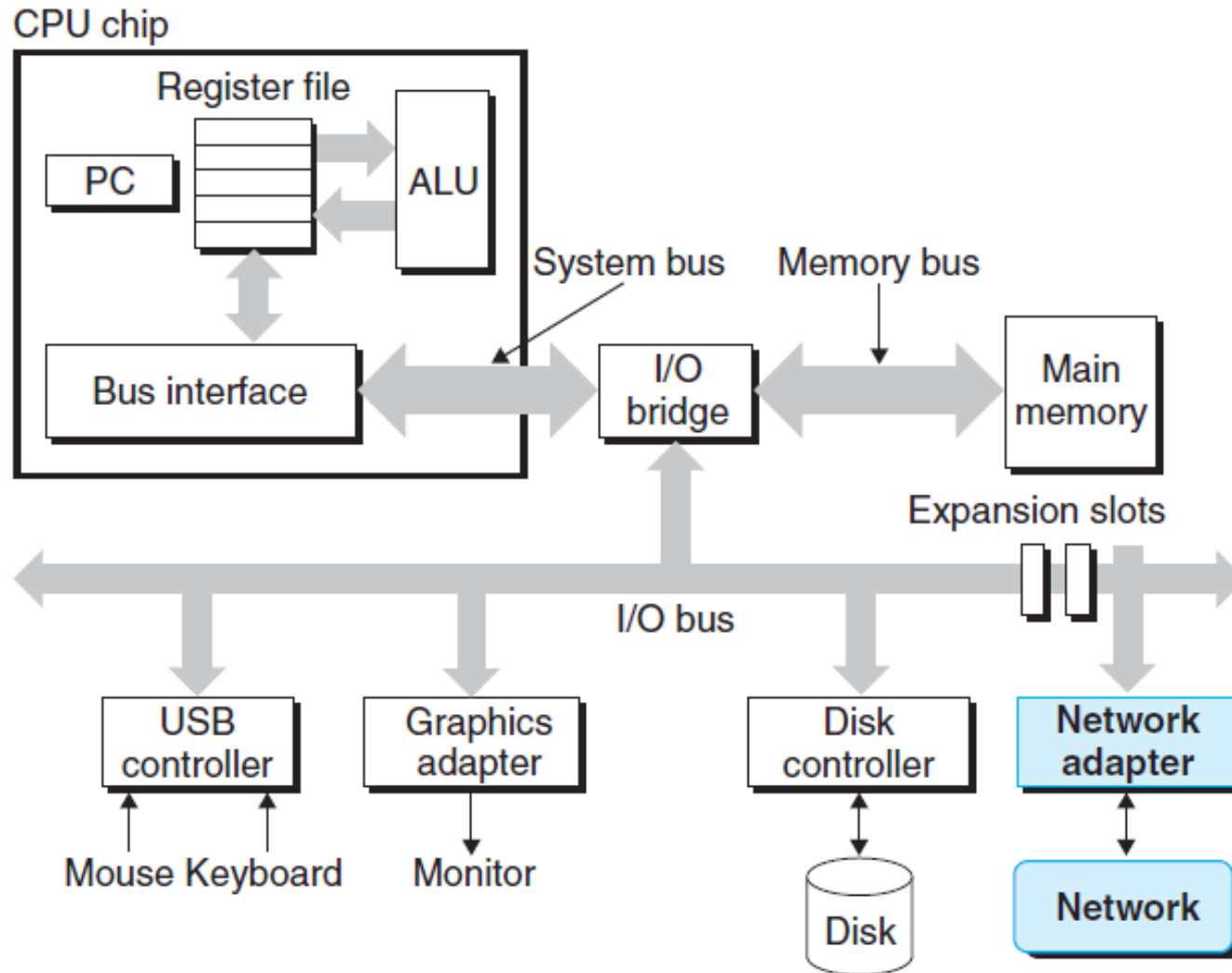
❑ Kernel Virtual Memory area

- The kernel is the part of the operating system that is always resident in memory.
- The top region of the address space is reserved for the kernel.
- Application programs are not allowed to read or write the contents of this area or to directly call functions defined in the kernel code.
- For virtual memory to work, a sophisticated interaction is required between the hardware and the operating system software, including a hardware translation of every address generated by the processor.
- The basic idea is to store the contents of a process's virtual memory on disk, and then use the main memory as a cache for the disk.

Network as an I/O device

- In practice, modern systems are often linked to other systems by networks.
- From the point of view of an individual system, the network can be viewed as just another I/O device
- When the system copies a sequence of bytes from main memory to the network adapter, the data flows across the network to another machine, instead of, say, to a local disk drive.
- Similarly, the system can read data sent from other machines and copy this data to its main memory.
- With the advent of global networks such as the Internet, copying information from one machine to another has become one of the most important uses of computer systems.
- For example, applications such as email, instant messaging, the World Wide Web, FTP, and telnet are all based on the ability to copy information over a network.

Network as an I/O Device



Network as another I/O Device

Run Program remotely over a Network

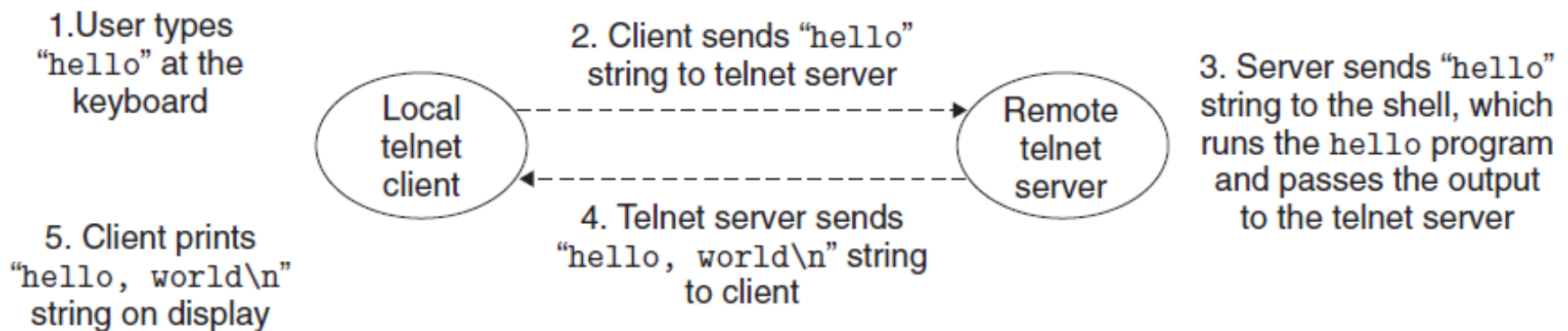
Consider a telnet application to run hello on a remote machine.

Suppose we use a telnet client running on our to the remote machine and run a shell, the remote shell is waiting to receive an input command.

Running the hello program remotely involves the five basic steps :

- After we type the “hello” string to the telnet client and hit the enter key, the client sends the string to the telnet server.
- After the telnet server receives the string from the network, it passes it along to the remote shell program.
- Next, the remote shell runs the hello program, and passes the output line back to the telnet server.
- Finally, the telnet server forwards the output string across the network to the telnet client
- Client then prints the output string on our local terminal.

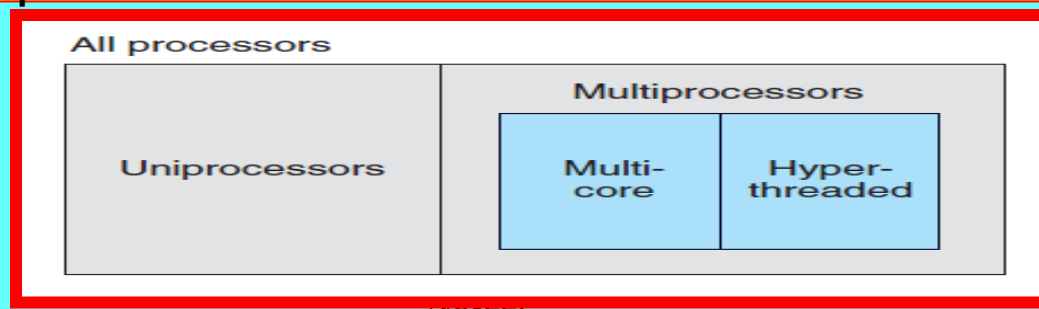
This type of exchange between clients and servers is typical of all network applications.



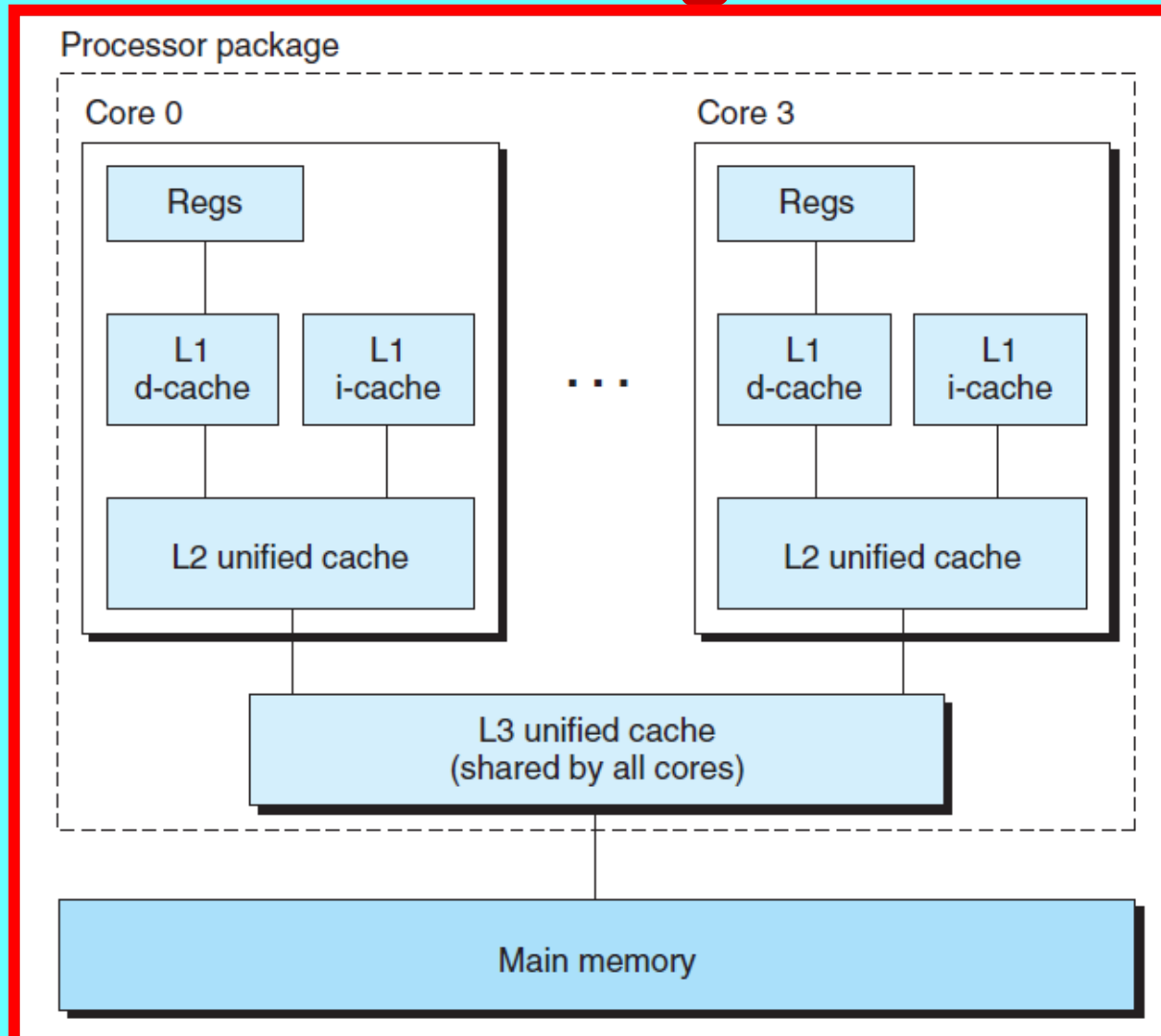
Using Telnet to run hello remotely over a network

Multi-Core Processors

- * Multi-core processors have several CPUs (referred to as “cores”) integrated onto a single integrated-circuit chip. In Intel Core i7 processor, the microprocessor chip has four CPU cores, each with its own L1 and L2 caches but sharing the higher levels of cache as well as the interface to main memory.
- * Industry experts predict that they will be able to have dozens, and ultimately hundreds, of cores on a single chip.
- * Hyperthreading, sometimes called simultaneous multi-threading, is a technique that allows a single CPU to execute multiple flows of control.
- * It involves having multiple copies of some of the CPU hardware, such as program counters and register files, while having only single copies of other parts of the hardware, such as the units that perform floating-point arithmetic. Whereas a conventional a hyperthreaded processor decides which of its threads to execute on a cycleby- cycle basis. It enables the CPU to make better advantage of its processing resources.
- * For example, if one thread must wait for some data to be loaded into a cache, the CPU can proceed with the execution of a different thread.
- * As an example, the Intel Core i7 processor can have each core executing two threads, and so a four-core system can actually execute eight threads in parallel.
- * The use of multiprocessing can improve system performance in two ways – First by reducing the need to simulate concurrency when performing multiple tasks and Second, by running a single application program faster, but only if that program is expressed in terms of multiple threads that can effectively execute in parallel.



Intel Core i7 Organization



Intel Core i7 Organization

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 1 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

SOC 2040

SYSTEM PROGRAMMING

END OF

WEEK 3 LECTURE 1

INTRODUCTION TO COMPUTER SYSTEM

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

